

CSE 60321 — Advanced Computer Architecture — Spring 2026

Homework 2

Assigned: Tuesday, February 10, 2026 – Due: Thursday, February 19, 2026

Scope of this submission. This document answers **Problems 1–4** ($21 + 10 + 8 + 16 = 55$ points). **Problem 5 (45 points) has been omitted intentionally**, at the explicit instruction of the submitter; it is not an oversight. Problem 5 is the Multi2Sim cache-configuration study, whose Questions B–F consist of execution-cycle measurements that must be gathered by running simulations on the Notre Dame CSE student machine `student04.cse.nd.edu`.

Each answer below is labelled with the problem and question it addresses, as required by the assignment’s instructions. The assignment’s binary unit convention ($1 \text{ KB} = 2^{10}$ Bytes, and so on) is used throughout.

Problem 1: Multi-Level Cache Hierarchy on an Intel Xeon Chip (21 points)

Conventions used throughout. The statement’s hint says that a miss penalty is an additional cost *on top of* the hit time, and that a miss is discovered only after the access to that level has been attempted. The hierarchy is therefore modelled by the nested expression

$$\text{AMAT} = HT_{L1} + MR_{L1} P_{L1}, \quad P_{L1} = HT_{L2} + MR_{L2} P_{L2}, \quad P_{L2} = HT_{L3} + MR_{L3} P_{L3}, \quad (1)$$

with $HT_{L1} = 1$, $HT_{L2} = 10$, $HT_{L3} = 50$ CC, and P_{L3} the main-memory access cost. The miss rates supplied in Questions C and D are *local* miss rates, which is exactly what (1) consumes, so no conversion to global rates is needed.

For the memory access itself, the initial access time M “accounts for a single transfer”, so with a block of 128 B and a bus of B bytes only the transfers after the first are charged 10 CC each:

$$P_{L3} = M + 10 \left(\frac{128 B}{B} - 1 \right). \quad (2)$$

Finally, the clock is 2.0 GHz, so $1 \text{ CC} = 1/(2.0 \times 10^9 \text{ Hz}) = 0.5 \text{ ns}$.

Question A (3 points)

- (1) **False.** The L1 caches hold only 64 KB each, so with L2 and L3 removed every L1 miss would have to be served by main memory (hundreds of cycles) instead of by a 10- or 50-cycle cache, which raises the average memory access time substantially.

- (2) **True.** L1 must be fast enough to answer in a single clock cycle, so it is built from faster, lower-density SRAM cells with more ports and more decoding logic per byte; L3 is latency-tolerant and can use denser, cheaper cells, so a given capacity costs more in L1 than in L3.
- (3) **False.** Caches are filled on demand, one block at a time, and the program resides in main memory; only the blocks actually being referenced need to be present in L1. Indeed a program larger than 64 KB could never fit in the L1 instruction cache at all, yet still executes correctly.

Question B (6 points)

Block offset. Entries are *Byte* addressable, so the offset field must select an individual byte within a block. With a 128-Byte block,

$$\# \text{offset bits} = \log_2 128 = 7.$$

(The fact that a data word is 64 bit = 8 Bytes does *not* shrink this field: because addressing is per Byte and not per word, the low 3 bits that select a byte inside a word are still part of the address and belong to the offset.)

Index. The L2 cache holds 512 KB = $512 \times 2^{10} = 2^{19}$ Bytes, so the number of blocks is

$$\# \text{blocks} = \frac{2^{19} \text{ B}}{128 \text{ B}} = \frac{2^{19}}{2^7} = 2^{12} = 4096.$$

The cache is 4-way set associative, i.e. 4 blocks per set, so

$$\# \text{sets} = \frac{4096}{4} = 1024 = 2^{10} \implies \# \text{index bits} = \log_2 1024 = 10.$$

Tag. The tag is whatever remains of the 64-bit physical address:

$$\# \text{tag bits} = 64 - \# \text{index} - \# \text{offset} = 64 - 10 - 7 = 47.$$

Answer.

Field	Tag	Index	Block offset
Width (bits)	47	10	7

Check: $47 + 10 + 7 = 64$ bits, as required.

Question C (10 points)

Local miss rates are $MR_{L1} = 5\%$, $MR_{L2} = 40\%$, $MR_{L3} = 20\%$, and the bus is 16 Bytes wide. A 128-Byte block therefore needs $128/16 = 8$ transfers, of which $8 - 1 = 7$ are *additional*, so by (2)

$$P_{L3} = M + 10 \times 7 = M + 70 \quad \text{CC},$$

where M is the initial main memory access time in CCs. Substituting into (1), the requirement $\text{AMAT} \leq 4$ CC reads

$$1 + 0.05 \left[10 + 0.40(50 + 0.20(M + 70)) \right] \leq 4.$$

Unwinding the nesting one level at a time:

$$\begin{aligned}
 P_{L1} &\leq \frac{4 - 1}{0.05} = 60 \text{ CC}, \\
 P_{L2} &\leq \frac{60 - 10}{0.40} = 125 \text{ CC}, \\
 P_{L3} &\leq \frac{125 - 50}{0.20} = 375 \text{ CC}, \\
 M &\leq 375 - 70 = 305 \text{ CC}.
 \end{aligned}$$

Conversion. At 2.0 GHz one clock cycle is 0.5 ns, hence

$$M_{\max} = 305 \text{ CC} \times 0.5 \frac{\text{ns}}{\text{CC}} = 152.5 \text{ ns}$$

Check. With $M = 305 \text{ CC}$: $P_{L3} = 375$, $P_{L2} = 50 + 0.2(375) = 125$, $P_{L1} = 10 + 0.4(125) = 60$, and $\text{AMAT} = 1 + 0.05(60) = 4 \text{ CC}$ exactly, so the bound is attained and is tight.

Question D (2 points)

The miss rates are unchanged and the initial main memory access time carried forward is the maximum value found in Question C, $M = 305 \text{ CC}$ ($= 152.5 \text{ ns}$). Only the bus width changes, from 16 to 32 Bytes, so a 128-Byte block now takes $128/32 = 4$ transfers, of which $4 - 1 = 3$ are additional. By (2),

$$P_{L3} = 305 + 10 \times 3 = 335 \text{ CC}.$$

Working back up through (1):

$$\begin{aligned}
 P_{L2} &= 50 + 0.20 \times 335 = 117 \text{ CC}, \\
 P_{L1} &= 10 + 0.40 \times 117 = 56.8 \text{ CC}, \\
 \text{AMAT} &= 1 + 0.05 \times 56.8 = 3.84 \text{ CC}.
 \end{aligned}$$

The average memory access time will be 3.84 CCs (equivalently 1.92 ns). As expected, doubling the bus width removes 40 CC of transfer time from the L3 miss penalty and lowers the AMAT below the 4 CC budget of Question C.

Problem 2: Virtual Memory Address Translation and Page Table Sizing (10 points)

Throughout, the assignment's binary unit convention applies, so $64 \text{ KB} = 2^{16} \text{ Bytes}$ and $2 \text{ GB} = 2^{31} \text{ Bytes}$.

Question A (2 points)

The page offset must address every byte within a page. A page holds $64 \text{ KB} = 2^{16} \text{ Bytes}$, so the offset is 16 bits, and the remaining $30 - 16 = 14$ bits of the 30-bit virtual address form the virtual page number.

Table 1: Virtual address components (Table 2 of the assignment)

Page offset / bits	Virtual page memory (VPN) / bits	Total / bits
16	14	30

Question B (4 points)

A single-level page table holds one entry per virtual page, so it is indexed by the 14-bit VPN from Question A; the physical memory size does not determine the number of entries.

Number of PTEs: $2^{14} = 16,384$ entries.

Physical memory for the page table: $2^{14} \times 4 \text{ Bytes} = 2^{16} \text{ Bytes} = 64 \text{ KB}$.

Question C (4 points)

Pros.

1. *Smaller page table.* A larger page size widens the offset field and narrows the VPN by the same number of bits, so the entry count falls geometrically. Doubling the page size in Question B to 128 KB would leave a 13-bit VPN, halving the table from 64 KB to 32 KB.
2. *Greater TLB reach.* Each TLB entry maps a whole page, so with larger pages a fixed number of TLB entries covers proportionally more of the address space. That lowers the TLB miss rate for programs with large working sets, and each avoided miss avoids a page-table walk.

Cons.

1. *More internal fragmentation.* Memory is allocated in whole pages, so the unused remainder of a partially filled page is wasted. The waste per incompletely filled region grows with the page size, which hurts most for many small objects or sparse address spaces.
2. *Costlier page faults.* A fault transfers an entire page from disk, so a larger page means a longer transfer and more memory traffic. If the program only touches a small part of that page, the extra bytes fetched are wasted I/O and the fault latency rises.

Problem 3: Way Prediction on an L1 Instruction Cache (8 points)

Throughout, we use the standard average-memory-access-time expression, in which the miss penalty is an *additional* cost incurred on top of the access time:

$$\text{AMAT} = \text{access time} + \text{miss rate} \times \text{miss penalty}. \quad (3)$$

The stated geometry (4-way set associative, 64 B blocks, 256 KB capacity) fixes the baseline 1.1 ns access time and the direct-mapped 1 ns comparison point; it is not needed numerically below.

Question A (2 points)

With way prediction the cache first probes the single predicted way. If that prediction is correct the access completes in the direct-mapped access time of 1 ns. If it is wrong, the 1.1 ns quoted for a misprediction is *additional* time, so the failed probe is followed by the ordinary associative lookup and the two costs add:

$$t_{\text{best}} = 1 \text{ ns}, \quad (4)$$

$$t_{\text{worst}} = 1 + 1.1 = 2.1 \text{ ns}. \quad (5)$$

Answer. For the best case a cache access takes **1 ns**; for the worst case it takes **2.1 ns**.

Here “best/worst case” is read as the two *cache-access* (hit-time) outcomes of the way predictor, namely prediction success versus prediction failure, since the 50 ns miss penalty is accounted separately in Eq. (3) rather than as part of the access time.

Question B (6 points)

Step 1: baseline AMAT (no way prediction). The original 4-way cache has a 1.1 ns access time, a 1% miss rate and a 50 ns additional miss penalty:

$$\text{AMAT}_{\text{base}} = 1.1 + 0.01 \times 50 = 1.1 + 0.5 = 1.6 \text{ ns}. \quad (6)$$

Step 2: AMAT with way prediction. Let a be the prediction accuracy. The prediction succeeds with probability a (costing $t_{\text{best}} = 1$ ns) and fails with probability $1 - a$ (costing $t_{\text{worst}} = 2.1$ ns), so the expected access time is $a(1) + (1 - a)(2.1)$. Since the miss rate is unchanged at 1% and the miss penalty is still 50 ns, the miss term is identical to the baseline:

$$\begin{aligned} \text{AMAT}_{\text{wp}}(a) &= [a \times 1 + (1 - a) \times 2.1] + 0.01 \times 50 \\ &= (2.1 - 1.1a) + 0.5 \\ &= 2.6 - 1.1a \text{ ns}. \end{aligned} \quad (7)$$

Step 3: impose the 5% reduction. Reducing the AMAT by at least 5% means the new AMAT is at most 95% of the baseline:

$$\text{AMAT}_{\text{wp}}(a) \leq 0.95 \times \text{AMAT}_{\text{base}} = 0.95 \times 1.6 = 1.52 \text{ ns}. \quad (8)$$

Step 4: solve for a . Substituting Eq. (7) into Eq. (8):

$$\begin{aligned} 2.6 - 1.1a &\leq 1.52 \\ 1.1a &\geq 2.6 - 1.52 = 1.08 \\ a &\geq \frac{1.08}{1.1} = \frac{108}{110} = \frac{54}{55} = 0.9818\overline{18}. \end{aligned} \quad (9)$$

Answer. The prediction accuracy must be *at least*

$$a_{\min} = \frac{54}{55} = 98.\overline{18}\% \approx \mathbf{98.19\%}. \quad (10)$$

Because this is a lower bound, the decimal answer is rounded *up*: the exact threshold $98.1818\dots\%$ rounded down to 98.18% would give $\text{AMAT}_{\text{wp}} = 2.6 - 1.1(0.9818) = 1.52002 \text{ ns}$, which exceeds the 1.52 ns budget and therefore fails the requirement, whereas 98.19% gives 1.51991 ns and satisfies it.

Check. At $a = 54/55$ exactly, $\text{AMAT}_{\text{wp}} = 2.6 - 1.1 \times \frac{54}{55} = 2.6 - 1.08 = 1.52 \text{ ns}$, which is exactly $0.95 \times 1.6 \text{ ns}$, i.e. precisely a 5% reduction, as required at the threshold. Note also that even a perfect predictor ($a = 1$) only reaches $\text{AMAT} = 1.5 \text{ ns}$, a 6.25% reduction, so the 5% target leaves very little margin — which is why the required accuracy is so high.

Problem 4: Critical-Word First and Early Restart (16 points)

Common setup and notation. A word is 8 Bytes, so a block of B Bytes holds $n = B/8$ words. For a given cache level write

t_1 time until the *first* word arrives when the block is sent in normal order (no critical-word first);

t_x time for each *additional* word;

t_c time until the requested word arrives when critical-word first is used (it is sent first, so this single transfer is all the processor waits for).

Without reordering, word i of the block ($i = 0, 1, \dots, n-1$) is available after $t_1 + i t_x$. The three schemes are then

$$\text{neither: } T_{\text{none}} = t_1 + (n-1) t_x \quad (\text{wait for the whole block}) \quad (11)$$

$$\text{early restart only: } T_{\text{ER}} = \sum_{i=0}^{n-1} p_i (t_1 + i t_x) \quad (\text{resume when word } i \text{ arrives}) \quad (12)$$

$$\text{both: } T_{\text{both}} = t_c \quad (\text{requested word first, then resume}) \quad (13)$$

where p_i is the probability that the referenced word sits at position i . Equations (11) and (13) do not involve p_i : in the first the processor waits for the last word regardless of what it asked for, and in the third the requested word is always the first one delivered. Only (12) is a genuine expectation over the reference distribution.

Question A (8 points)

Here $B = 64$ Bytes, so

$$n = \frac{64 \text{ Bytes}}{8 \text{ Bytes/word}} = 8 \text{ words per block},$$

with $t_1 = 90$, $t_x = 10$ and $t_c = 110$ CCs. (The 1 MB capacity is not needed: the miss service time depends on the block size, not on how many blocks the cache holds.) Word i therefore arrives at $90 + 10i$ CCs, i.e. at 90, 100, 110, 120, 130, 140, 150, 160 CCs for $i = 0, \dots, 7$.

The reference distribution is *not* uniform. The first word carries 30% and the remaining $n - 1 = 7$ words share the other 70% equally:

$$p_0 = 0.30, \quad p_i = \frac{0.70}{7} = 0.10 \quad (i = 1, \dots, 7), \quad \text{check: } 0.30 + 7(0.10) = 1.$$

(1) Neither critical-word first nor early restart. The processor resumes only once the entire block has arrived, which happens when the last word lands:

$$T_{\text{none}} = t_1 + (n - 1)t_x = 90 + 7 \times 10 = \boxed{160 \text{ CCs}}.$$

This does not depend on which word was requested, so the reference distribution is irrelevant here and the “average” equals this fixed value.

(2) Early restart only. The block still arrives in normal order, but the processor resumes as soon as the requested word has arrived, so the wait depends on that word’s position:

$$\begin{aligned} T_{\text{ER}} &= 0.30 \times 90 + 0.10(100 + 110 + 120 + 130 + 140 + 150 + 160) \\ &= 27 + 0.10 \times 910 = 27 + 91 = \boxed{118 \text{ CCs}}. \end{aligned}$$

(3) Both critical-word first and early restart. Memory delivers the requested word first, at a cost of $t_c = 110$ CCs, and the processor restarts immediately:

$$T_{\text{both}} = 110 \text{ CCs} \Rightarrow \boxed{110 \text{ CCs}}.$$

Again the block-position distribution drops out entirely: whichever word is requested, it is the one sent first, so every miss costs the same 110 CCs. Note that critical-word first is *more expensive per transfer* than a normal first word ($110 > 90$); it still wins overall because it removes the wait for all seven remaining words.

Summary.

Scheme	Average L2 miss service time	Saving vs. (1)
(1) Neither	160 CCs	—
(2) Early restart only	118 CCs	$42/160 = 26.25\%$
(3) Critical-word first + early restart	110 CCs	$50/160 = 31.25\%$

Question B (4 points)

Answers: the first blank is 18 CCs and the second blank is 10%.

Now $B = 32$ Bytes, so

$$n = \frac{32 \text{ Bytes}}{8 \text{ Bytes/word}} = 4 \text{ words per block},$$

with $t_x = 4$ CCs, $t_c = 27$ CCs, and every word equally likely, $p_i = \frac{1}{4}$. Let x denote the unknown first-word latency.

First blank — recovered from the stated 20%. The paragraph gives us the early-restart result and asks us to work backwards. In terms of x ,

$$T_{\text{none}} = x + (n - 1)t_x = x + 3 \times 4 = x + 12,$$

$$T_{\text{ER}} = \frac{1}{4}[x + (x + 4) + (x + 8) + (x + 12)] = x + \frac{4(0 + 1 + 2 + 3)}{4} = x + 6.$$

Early restart alone reduces the original time by 20%, so

$$\frac{T_{\text{none}} - T_{\text{ER}}}{T_{\text{none}}} = \frac{(x + 12) - (x + 6)}{x + 12} = \frac{6}{x + 12} = 0.20 \implies x + 12 = 30 \implies \boxed{x = 18 \text{ CCs}}.$$

Hence $T_{\text{none}} = 30$ CCs and $T_{\text{ER}} = 24$ CCs, which indeed differ by $6/30 = 20\%$.

Second blank. With critical-word first the requested word arrives after $t_c = 27$ CCs and the processor restarts there, so $T_{\text{both}} = 27$ CCs and

$$\frac{T_{\text{none}} - T_{\text{both}}}{T_{\text{none}}} = \frac{30 - 27}{30} = \frac{1}{10} = \boxed{10\%}.$$

Both blanks are exact integers, as the question requires.

Question C (4 points)

Verdict. On these numbers the two techniques are **more important for the L2 cache**.

Reason (from the results of A and B). Comparing the savings the two techniques actually deliver at each level:

	L2 (Question A)	L1 (Question B)
Words per block	8	4
Neither	160 CCs	30 CCs
Early restart only	118 CCs (−26.25%)	24 CCs (−20%)
Critical-word first + early restart	110 CCs (−31.25%)	27 CCs (−10%)
Absolute saving of the pair	50 CCs	3 CCs

At L2 the pair of techniques removes 31.25% of the miss service time (50 CCs); at L1 it removes only 10% (3 CCs). The gap is even sharper for critical-word first taken on its own: at L2 it improves on early restart alone (118 → 110 CCs), whereas at L1 it makes matters *worse* (24 → 27 CCs), because its $27 - 18 = 9$ CC surcharge exceeds the 6 CC that early restart had already saved. So at L1 it would be better not to enable critical-word first at all, while at L2 it pays for itself — which is precisely what “more important for L2” means.

Factors that drive the relative importance.

1. *Words per block, n .* The most these techniques can ever save is the transfer tail $(n - 1)t_x$ — the time spent shipping the words the processor is not waiting for. Larger blocks mean a longer tail and more to skip. Lower cache levels normally use larger blocks (8 words here versus 4), which favours L2.

2. *Fixed first-word latency versus the tail.* The ceiling on the saving is $(n-1)t_x/[t_1 + (n-1)t_x]$: $70/160 = 43.75\%$ at L2 and $12/30 = 40\%$ at L1. When the fixed latency dominates, the processor must wait for it anyway and neither technique can help much. Here the two levels start out comparable, so this is not what separates them.
3. *The surcharge critical-word first itself costs.* Reordering the transfer is not free: $t_c - t_1$ is 20 CCs at L2 but 9 CCs at L1. Relative to the block service time that is $20/160 = 12.5\%$ at L2 against $9/30 = 30\%$ at L1. This is the decisive factor: it consumes 12.5 of L2's 43.75 available points but 30 of L1's 40, leaving 31.25% and 10% respectively. The shorter a level's total miss time, the harder a fixed reordering overhead is to justify.
4. *Shape of the reference distribution.* Early restart only pays off to the extent that the requested word arrives early. L2's 30% weight on the first word helps it: under a uniform $1/8$ the L2 early-restart average would be 125 CCs, a saving of 21.875% rather than 26.25%. Critical-word first is immune to this, since its cost is flat in the word position — which makes it the more attractive of the two when the distribution is unfavourable or unknown.
5. *Miss frequency (a full-AMAT caveat).* The comparison above is *per miss*, which is what this problem measures. A complete average-memory- access-time analysis would also weight each level's saving by how often that level misses, and L1 misses are far more frequent than L2 misses. That is the one factor that could push the balance back towards L1; with the numbers given here, though, L1's saving is only 3 CCs against L2's 50 CCs, so it would take an L1:L2 miss ratio above roughly 16:1 merely to draw level.